WAFT Architecture - One-Pager

WAFT Architecture Investigation

Generated: 2026-01-11 16:43:15

Work Effort: WE-260111-2i9f

Entry Point: README.md

Status: 🔍 Investigation Complete

WAFT Architecture Investigation

Date: 2026-01-11

Entry Point: `README.md`

Status: 🔍 In Progress

Investigation Methodology

Starting from `README.md` as the entry point, tracing through:

1. CLI entry point (`src/waft/main.py`)
2. Core module structure
3. Component relationships
4. Design patterns and architecture decisions

---

1. Entry Point Analysis: README.md

Core Purpose

WAFT: "A Python framework for directed evolution of self-modifying AI agents"

- **Goal**: Observe a "God-Head" agent emerge from evolutionary process
- **Mission**: Produce data for "The Physics of Artificial Cognition" research

Three Core Pillars (from README)

1. The Substrate

- Agents write their own Python source code (DNA)
- Code is DNA - agents can spawn, evolve, reproduce
- Genome ID = SHA-256 hash of code + configuration
- Mutations = modifications to genome

2. The Physics

- **Scint System** (Reality Fracture Detection) = fitness function

- **Scint Gym** tests agents on:
- SYNTAX_TEAR: Formatting errors
- LOGIC_FRACTURE: Math errors, contradictions
- SAFETY_VOID: Harmful content, PII leaks
- HALLUCINATION: Fabricated facts
- Fitness = Stability (40%) + Efficiency (30%) + Safety (30%)
- Fitness < 0.5 = DEATH (evolutionary dead end)

3. The Flight Recorder

- Complete telemetry for phylogenetic trees
- Records: Genome ID, Parent ID, Generation, Event Type, Payload, Fitness Metrics
- Enables: Family trees, mutation impact, fitness landscape, convergence analysis

---

2. CLI Entry Point: `src/waft/main.py`

Main Application Structure

Framework: Typer (CLI framework)

Entry Point: `main()` function → `app()` (Typer app)

Core Managers (Orchestration Layer)

From `main.py` imports:

```
from .core.memory import MemoryManager           # _pyrite memory system
from .core.substrate import SubstrateManager     # Agent substrate
from .core.empirica import EmpiricaManager       # Epistemic tracking
from .core.gamification import GamificationManager  # D&D gamification
from .core.github import GitHubManager           # GitHub integration
from .core.tavern_keeper import TavernKeeper     # Narrative system
```

Command Structure

Main Commands (direct on `app`):

- `new` - Create new laboratory
- `verify` - Verify project structure
- `evolve` - Run evolutionary cycle (Coming Soon)
- `sync` - Sync dependencies (uv sync)
- `add` - Add dependency
- `init` - Initialize WAFT structure
- `info` - Show project info
- `serve` - Web dashboard (FastAPI + SvelteKit)
- `decide` - Decision matrix analysis

Sub-Commands (Typer sub-apps):

- `session` - Empirica session management
- `finding` - Finding logging
- `unknown` - Unknown logging
- `goal` - Goal management
- `github` - GitHub integration
- `journal` - Development journal
- `analytics` - Analytics commands

Gamification Commands:

- `dashboard` - Epistemic HUD
- `stats` - Current stats
- `character` - D&D character sheet
- `chronicle` - Adventure journal
- `observe` - Log observations

Tavern Keeper Integration

Hook System: *process*`tavern_hook()` processes command hooks

- Narrative generation
- Dice roll mechanics (D&D ability checks)
- XP rewards and leveling
- Command classification (critical*success, critical*failure, etc.)

---

3. Core Module Structure

`src/waft/core/` - Core Orchestration

Key Modules:

- `memory.py` - MemoryManager (manages `_pyrite/` structure)
- `substrate.py` - SubstrateManager (agent substrate system)
- `empirica.py` - EmpiricaManager (epistemic tracking)
- `gamification.py` - GamificationManager (D&D mechanics)
- `github.py` - GitHubManager (GitHub integration)
- `tavern_keeper/` - Narrative and gamification system
- `keeper.py` - TavernKeeper main class
- `narrator.py` - Narrative generation
- `grammars.py` - Narrative grammars
- `ai_helper.py` - AI helper integration

Agent System (`core/agent/`):

- `base.py` - BaseAgent class

- `state.py` - Agent state management
- `anatomy.py` - Agent anatomy/configuration
- `items.py` - Agent items/inventory

Science System (`core/science/`):

- `taxonomy.py` - LineagePoet (scientific naming)
- `lab_entry.py` - LabEntryGenerator
- `notebook.py` - Scientific notebook
- `observer.py` - Observer system
- `report.py` - Scientific reports
- `tam_psyche.py` - TAM psyche system

Decision System:

- `decision_cli.py` - DecisionCLI
- `decision_matrix.py` - DecisionMatrixCalculator
- `workflow_decision.py` - WorkflowDecisionAnalyzer

Other Core Modules:

- `analytics_cli.py` - Analytics commands
- `audit.py` - Audit system
- `checkout.py` - Checkout system
- `continue_work.py` - Continue work system
- `goal.py` - Goal management
- `help.py` - Help system
- `persistence.py` - Persistence layer
- `proceed.py` - Proceed system
- `recap.py` - Recap generation
- `reflect.py` - Reflection system
- `resume.py` - Resume system
- `session_analytics.py` - Session analytics
- `session_stats.py` - Session statistics
- `visualizer.py` - Visualizer
- `workflow.py` - Workflow system

`src/waft/evolution/` - Evolution System

Key Modules:

- `chat_distiller.py` - ChatDistiller (extract ideas from conversations)
- `styling_genome.py` - StylingGenome (document styling as genetic material)
- `twopagegenerator.py` - TwoPageGenerator (2-page PDF generation)
- `pdf_generator.py` - PDFGenerator (composable PDF generation)

- `scientific_pdf_generator.py` - ScientificPDFGenerator (self-examination)
- `latex_generator.py` - LaTeXGenerator (LaTeX document generation)
- `component_generator.py` - ComponentPDFGenerator (component-based PDFs)
- `document_evolution_engine.py` - DocumentEvolutionEngine
- `scint_detector.py` - ScintDetector (reality fracture detection)
- `pdf_research_tool.py` - PDFResearchTool (cross-PDF analysis)
- `pdf_metrics.py` - PDFMetrics, PDFMetricsCollector

Templates:

- `templates/scientific_research_paper.md` - Scientific paper template

`src/waft/templates/` - Document Templates

- `field_guide.py` - Field guide template
- `lab_notes.py` - Lab notebook template
- `personal_memo.py` - Personal memo template
- `tm_report.py` - Technical memo template
- `one_pager.py` - One-pager template

`src/waft/api/` - Web API (FastAPI)

Structure:

- `main.py` - FastAPI app creation
- `models.py` - API models
- `routes/` - API routes
- `decision.py` - Decision endpoints
- `empirica.py` - Empirica endpoints
- `git.py` - Git endpoints
- `gym.py` - Gym endpoints
- `state.py` - State endpoints
- `work_efforts.py` - Work efforts endpoints

`src/waft/cli/` - CLI Display Components

- `epistemic_display.py` - Epistemic dashboard display
- `hud.py` - HUD rendering

`src/waft/ui/` - UI Components

- `dashboard.py` - RedOctoberDashboard (web dashboard)

---

4. Component Relationships

Data Flow

```
User Command (CLI)
    ↓
main.py (Typer app)
    ↓
Core Manager (MemoryManager, SubstrateManager, etc.)
    ↓
Core Module (memory.py, substrate.py, etc.)
    ↓
Data Storage (_pyrite/, _genetics/, etc.)
```

Evolution System Flow

```
Content/Conversation
    ↓
ChatDistiller → DistilledChat (ideas extracted)
    ↓
StylingGenome → Styling configuration
    ↓
TwoPageGenerator / PDFGenerator / LaTeXGenerator
    ↓
Output (PDF, LaTeX, HTML)
    ↓
PDFMetrics / PDFResearchTool (analysis)
```

Agent System Flow

```
Agent Definition (src/agents.py)
    ↓
SubstrateManager → Agent substrate
    ↓
Agent Execution → Code generation/modification
    ↓
ScintDetector → Reality fracture detection
    ↓
Fitness Evaluation → Stability + Efficiency + Safety
    ↓
Flight Recorder → Evolutionary events logged
```

---

5. Key Design Patterns

1. Manager Pattern

- **MemoryManager**: Manages `_pyrite/` structure
- **SubstrateManager**: Manages agent substrate
- **EmpiricaManager**: Manages epistemic tracking
- **GamificationManager**: Manages D&D mechanics

- **GitHubManager**: Manages GitHub integration

2. Generator Pattern

- **PDFGenerator**: Composable PDF generation
- **LaTeXGenerator**: LaTeX document generation
- **ChatDistiller**: Content distillation
- **ComponentPDFGenerator**: Component-based generation

3. Genome Pattern

- **StylingGenome**: Document styling as genetic material
- **Genome ID**: SHA-256 hash for unique identification
- **Lineage Tracking**: Parent-child relationships
- **Evolutionary Events**: Flight recorder for all changes

4. Template Pattern

- **Template System**: Jinja2 templates for document generation
- **WeasyPrint**: HTML → PDF conversion
- **Style Presets**: clinical_standard, premium, professional

5. Hook Pattern

- **Tavern Keeper Hooks**: Command hooks for narrative/gamification
- **Command Processing**: `process_tavern_hook()` processes all commands
- **Dice Rolls**: D&D ability checks for commands
- **Narrative Generation**: Context-aware narratives

---

6. Storage Structure

`_pyrite/` - Memory System

- `active/` - Current work
- `backlog/` - Future work
- `standards/` - Standards
- `gym_logs/` - Scint Gym results

`_genetics/` - Genetic Material

- `pdf_generator/` - PDF generator genomes
- Evolutionary event tracking

`_workefforts/` - Work Tracking

- Work effort documentation
- Johnny Decimal organization

---

7. Integration Points

External Systems

- **uv**: Package management
- **Empirica**: Epistemic tracking
- **GitHub**: Repository management
- **CrewAI**: Agent framework (mentioned in main.py)
- **WeasyPrint**: PDF generation
- **FastAPI**: Web API
- **SvelteKit**: Web frontend

Internal Systems

- **Evolution System**: Document generation and evolution
- **Gamification System**: D&D mechanics and narrative
- **Memory System**: `_pyrite/` structure
- **Agent System**: Self-modifying agents
- **Scint System**: Reality fracture detection

---

8. Agent System Architecture

BaseAgent Class (`src/waft/core/agent/base.py`)

Core Concept: "The Organism" - self-modifying AI agents with biological lifecycle

Key Features:

- **Genome ID**: SHA-256 hash of agent configuration + code
- **Lineage Tracking**: parent*id, generation, lineage*path
- **Flight Recorder**: Complete event history
- **Scientific Naming**: LineagePoet generates scientific names
- **Anatomical Archetype**: Deterministic symbol assignment

Biological Lifecycle:

1. **Spawn**: Create child agent with mutation
2. **Eval**: Test fitness in Scint Gym
3. **Evolve**: Hot-swap to better variant

OODA Loop (Abstract Methods):

- `observe()` - Observe current project state
- `decide()` - Make decision using decision engine
- `act()` - Execute action
- `reflect()` - Reflect on outcome and learn

State Management:

- `AgentState` - Pydantic model for type safety
- `AgentConfig` - Agent configuration
- `EvolutionaryEvent` - Event recording
- `Modification` - Self-modification request

Memory System (`src/waft/core/memory.py`)

MemoryManager: Manages `_pyrite/` structure

Structure:

- `active/` - Current work
- `backlog/` - Future work
- `standards/` - Project standards

Methods:

- `create*structure()*` - *Create* pyrite directory structure
- `verify_structure()` - Verify structure validity
- `get*active*files()` - Get active files
- `get*backlog*files()` - Get backlog files

---

9. Evolution System Architecture

Document Generation Flow

```
Content/Conversation
    ↓
ChatDistiller.distill_text()
    ↓
DistilledChat (ideas extracted as IdeaGene)
    ↓
StylingGenome (styling configuration)
    ↓
Generator (TwoPageGenerator / PDFGenerator / LaTeXGenerator)
    ↓
Output (PDF, LaTeX, HTML)
```

Key Components

ChatDistiller (`evolution/chat_distiller.py`):

- Extracts ideas from conversations
- Creates IdeaGene objects (genetic material)
- Categories: concept, decision, insight, action, question

StylingGenome (`evolution/styling_genome.py`):

- Document styling as genetic material

- Genes: FontGene, MarginGene, ColorGene, LayoutGene
- Genome ID = SHA-256 hash of styling configuration
- Enables evolution of document design

TwoPageGenerator (`evolution/twopagegenerator.py`):

- Adaptive 2-page PDF generation
- Real page counting with WeasyPrint
- Constraint satisfaction (exactly 2 pages)
- Fitness evaluation

PDFGenerator (`evolution/pdf_generator.py`):

- Composable PDF generation API
- Presets: clinical_standard, premium, professional
- Uses ChatDistiller + StylingGenome + TwoPageGenerator

LaTeXGenerator (`evolution/latex_generator.py`):

- LaTeX document generation
- Markdown to LaTeX conversion
- Integration with ChatDistiller and StylingGenome
- Optional PDF compilation

---

10. Design Patterns Identified

1. Manager Pattern

Purpose: Orchestration layer for major subsystems

- MemoryManager, SubstrateManager, EmpiricaManager, etc.
- Single responsibility per manager
- Consistent interface pattern

2. Generator Pattern

Purpose: Document/content generation

- PDFGenerator, LaTeXGenerator, ComponentPDFGenerator
- Builder pattern with fluent interface
- Preset-based configuration

3. Genome Pattern

Purpose: Genetic material representation

- StylingGenome, Agent genome (BaseAgent)
- Genome ID = SHA-256 hash for uniqueness
- Lineage tracking (parent*id, generation, lineage*path)
- Scientific naming via LineagePoet

4. Distiller Pattern

Purpose: Content extraction and structuring

- ChatDistiller extracts ideas from conversations
- Creates structured data (DistilledChat, IdeaGene)
- Enables downstream processing

5. Template Pattern

Purpose: Document generation templates

- Jinja2 templates for HTML generation
- WeasyPrint for HTML → PDF
- Style presets for consistent output

6. Hook Pattern

Purpose: Command processing and gamification

- TavernKeeper hooks process all commands
- Narrative generation
- D&D mechanics (dice rolls, XP, leveling)

7. Observer Pattern

Purpose: Event tracking and telemetry

- TheObserver (singleton) tracks all events
- Flight Recorder records evolutionary events
- Enables phylogenetic tree reconstruction

---

11. Data Flow Patterns

Command Execution Flow

```
User Command (CLI)
    ↓
main.py (Typer app)
    ↓
Command Handler
    ↓
Core Manager (MemoryManager, SubstrateManager, etc.)
    ↓
Core Module (memory.py, substrate.py, etc.)
    ↓
Data Storage (_pyrite/, _genetics/, etc.)
    ↓
TavernKeeper Hook (narrative, gamification)
    ↓
Response to User
```

Agent Evolution Flow

```
Agent Definition
    ↓
BaseAgent.__init__() → Compute genome_id
    ↓
Agent Execution (OODA loop)
    ↓
Self-Modification Request
    ↓
Spawn Child with Mutation
    ↓
Fitness Evaluation (Scint Gym)
    ↓
Evolve to Better Variant
    ↓
Flight Recorder (event logging)
    ↓
TheObserver (telemetry)
```

Document Generation Flow

```
Content (Markdown/Text)
    ↓
ChatDistiller → DistilledChat
    ↓
StylingGenome → Styling Configuration
    ↓
Generator (PDF/LaTeX/Component)
    ↓
Output (PDF/LaTeX/HTML)
    ↓
PDFMetrics / PDFResearchTool (analysis)
```

Next Steps

1. ✅ Read and analyze root README.md
2. ✅ Identify main entry points and core modules
3. 🔄 Map component relationships and dependencies (in progress)
4. ⏳ Document architecture patterns and design decisions
5. ⏳ Generate architecture documentation using WAFT tools

Status: Investigation in progress - documenting findings as we go.

Investigation Summary

This architecture investigation started from the README.md entry point and traced through:

1. ✅ CLI entry point (main.py)
2. ✅ Core module structure
3. ✅ Component relationships
4. ✅ Design patterns
5. ✅ Data flow patterns

Key Findings:

- WAFT is a self-modifying AI SDK with evolutionary architecture
- Three core pillars: Substrate, Physics (Scint System), Flight Recorder
- Manager pattern for orchestration
- Genome pattern for genetic material representation
- Generator pattern for document generation
- Complete lineage tracking for scientific research

Documentation Generated Using WAFT Tools:

- PDFGenerator for PDF documentation
- LaTeXGenerator for LaTeX documentation
- OnePager for 2-page summary

This demonstrates "using WAFT to develop WAFT" - documenting architecture with WAFT's own tools!